

PPD - Laborator 2 - Documentatie

C++ - Dinamic

C++	Tip Matrice	Tip Filtru	Nr Threads	Tip Thread	Timp Executie
1)	N=M=10	n=m=3	-	Static	0.01
			2	Orizantal	0.91
2)	N=M=1000	n=m=3	-	Static	71.2
			2	Orizantal	38.7
			4	Orizantal	21.76
			6	Orizantal	16.41
			16	Orizantal	15.07
3)	N=10000 M=10000	n=m=3	-	Static	4577.76
			2	Orizantal	2117.2
			4	Orizantal	1384.17
			8	Orizantal	889.85
			16	Orizantal	697.39

Java

Java	Tip Matrice	Tip Filtru	Nr Threads	Tip Thread	Timp Executie
1)	N=M=10	n=m=3	-	Static	0.62
			2	Orizantal	3.70
2)	N=M=1000	n=m=3	-	Static	66.65
			2	Orizantal	70.95
			4	Orizantal	73.65
			8	Orizantal	103.08
			16	Orizantal	118.52
3)	N=10000 M=10000	n=m=3	-	Static	5361.53
			2	Orizantal	1647.38
			4	Orizantal	1829.29
			8	Orizantal	949.93
			16	Orizantal	744.01

1. Comparația Performanței pe Cazuri

- **10x10:** Paralelizarea eșuează total. Costurile de creare/sincronizare a thread-urilor (overhead) sunt mult mai mari decât timpul de calcul.
- **1000x1000:** C++ scalează corect, găsind un punct optim la 16 thread-uri. Java scalează negativ
- **10000x10000:** Paralelizarea este vitală, reducând timpul C++ .

2. Comparație C++ vs. Java

C++ este **semnificativ mai rapid** decât Java în aproape toate testele valide.

Pentru sarcini triviale, ambele implementări sunt **semnificativ mai lente** când sunt paralelizate. Costul de administrare a thread-urilor (creare, sincronizare la barieră, join) depășește masiv timpul de calcul, care este aproape instantaneu. Java are un cost de pornire (JVM) mai mare decât C++.

3. Complexitate-Spațiu

Complexitatea spațială este aceeași pentru ambele limbaje, fiind dominată de matricea de intrare.

- **Total:** $O(N \times M)$
- **Spațiu Suplimentar (Secvențial):** $O(M)$ (pentru 3 buffere de rânduri).
- **Spațiu Suplimentar (Paralel):** $O(P \times M)$ (buffere pentru P thread-uri).

4. Observatii

- Fata de tema precedenta, consider ca unele alocari suplimentare de timp se datoreaza alocarii s deacolarii frecvente , inainte fiind alocata o singura data intreaga matrice.
- Pentru lucrul cu bariera, pentru fiecare thread copiez inainte randul de deasupra si de sub secventa careia ii este asignata.
- Executia propriu zisa, calculele, incep abia dupa ce ne asiguram ca toate threadurile si-au copiat randurile necesare, randuri care apartin executiei altor threaduri.
- Sincronizarea a fost realizata cu ajutorul unei bariere.